

Lab Practical 08

Git, Collaborative Development, CI/CD, Security & Code Quality

Lab Practical Number	Lab 08
Lab Title	Git, Collaborative Development, CI/CD, Security & Code Quality
Related Lecture	Lecture 08 – Git, Collaborative Development, CI/CD, Security and Code Quality
Duration	2 Hours (in-lab)
Learning Outcomes	LO3 – Best practices for collaboration, CI/CD, quality & deployment • LO4 – Select & justify approaches
Tooling	Git • GitHub • GitHub Actions • GitHub Issues • npm audit / Dependabot
Mode	Individual work (pairs for code review) • Word document + repo link submitted to CourseWeb

Introduction

This lab is about how professional teams actually ship software. In the lecture you studied version control with Git and GitHub, branching and collaborative workflows, pull requests and code review, issue tracking, continuous integration and delivery (CI/CD), code quality, and secure development. In this 2-hour session you will run that whole workflow end-to-end: create a repository, branch, commit, open and merge a pull request, track work with Issues, add an automated CI pipeline with GitHub Actions, and finish with a short code-quality and security review.

Everything you need comes from Lecture 08. Base your Git commands, workflow decisions, CI configuration, code-quality improvements, and security explanations on the lecture content and the class examples (the CI/CD pipeline Commit → Build → Test → Deploy → Monitor, “failing checks block a merge”, and “never commit secrets”).

CONTINUITY WITH LABS 02–04

Across the earlier labs you built the CampusEats frontend and a secured backend. This lab adds the delivery layer that ties them together: shared version control, collaboration through pull requests, and an automated pipeline that checks every change.

Lab Objectives

By the end of this lab, you should be able to:

- **Set up and use a Git/GitHub repository** cloning it and running the core Git commands confidently (LO3).
- **Work on feature branches with clean commits** using a sensible branch-naming and commit-message convention (LO3, LO4).
- **Collaborate through pull requests and code review** opening, reviewing, and merging a PR into main (LO3).
- **Track work with GitHub Issues** and link commits or PRs to issues (LO3).

- **Build a simple CI pipeline with GitHub Actions** that runs automatically on push and pull request (LO3, LO4).
- **Improve code quality and apply secure practices** fixing clean-code issues, removing secrets, and checking dependencies (LO3, LO4).

Required Tools / Software

- **A GitHub account** — sign in at github.com before the lab.
- **Git** — installed locally; verify with `git --version`.
- **A terminal** — and a code editor (VS Code recommended).
- **A web browser** — for GitHub (repository, pull requests, Issues, Actions).
- **Optional: Node.js or the .NET SDK** — only if you extend the CI workflow to build your Lab 02/03 project.

ONE-TIME GIT SETUP

If you have not used Git on this machine, set your identity first:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Lab Time Plan (120 minutes)

Time	Activity	Output
0:00 – 0:10	Briefing & recap of Lecture 08 (Git, PRs, CI/CD, security)	Ready to start
0:10 – 0:25	Task 01 — GitHub repository setup + clone	Repo cloned locally
0:25 – 0:45	Task 02 — Branching and commit workflow	Feature branch pushed
0:45 – 1:05	Task 03 — Pull request and code review	PR merged to main
1:05 – 1:17	Task 04 — GitHub Issues and collaboration	Issues created
1:17 – 1:39	Task 05 — CI pipeline with GitHub Actions	Green workflow run
1:39 – 1:52	Task 06 — Code quality and security review	Improved code + reflection
1:52 – 2:00	Quick test + screenshots and submit	Submitted .docx

HOW TO USE THIS PLAN

These times are a guide. The Git workflow (Tasks 01–03) is the core — keep it moving so you have time for the CI pipeline in Task 05. Take screenshots as you go; you will need them for submission.

Mini Project Scenario

“CAMPUSEATS TASK TRACKER” — A TEAM REPOSITORY

The three CampusEats developers need to track their work and collaborate cleanly. You will create a small **CampusEats Task Tracker** repository, then practise the exact workflow a real team uses: branch → commit → pull request → review → merge, with Issues for planning and GitHub Actions checking every change.

The repository content is deliberately light — a README and a small file — so you can focus on the **workflow and automation**, not on writing a large application.

Suggested repository structure

```
campuseats-task-tracker/  
  README.md           # project overview  
  .gitignore          # ignored files (node_modules, bin, obj)  
  src/  
    tasks.js          # a small sample file you will improve  
  .github/  
    workflows/  
      ci.yml           # the CI pipeline (Task 05)
```

TASK 01 • 15 minutes • LO3

GitHub Repository Setup

Create the team repository on GitHub and bring a copy to your machine.

Step-by-step instructions

1. On GitHub, click **New** and create a **public** repository named `campuseats-task-tracker`. Tick **Add a README** and choose a `.gitignore` template (Node).
2. Copy the repository URL, then clone it locally and move into the folder:

Terminal

```
$ git clone https://github.com/<you>/campuseats-task-tracker.git
$ cd campuseats-task-tracker
$ git status          # should say: On branch main, clean
```

3. Create a small source file so there is something to work with:

Terminal

```
$ mkdir src
$ echo "// CampusEats task list" > src/tasks.js
```

4. Confirm the remote is connected (this is where `push` and `pull` go):

Terminal

```
$ git remote -v          # origin → your GitHub repo URL
```

EXPECTED OUTPUT

A public `campuseats-task-tracker` repository on GitHub, cloned locally. Screenshots of the GitHub repo page and your terminal showing `git status` and `git remote -v`.

TASK 02 • 20 minutes • LO3, LO4

Branching and Commit Workflow

Real teams never commit straight to `main`. You work on a short-lived **feature branch**, then merge it through a pull request. This is the **GitHub Flow** model from the lecture: `main` is always deployable; each change gets its own branch.

Step-by-step instructions

1. Create and switch to a feature branch in one step:

Terminal

```
$ git switch -c feature/add-task-list
# (older syntax: git checkout -b feature/add-task-list)
```

2. Edit `src/tasks.js` — add a few task items, for example:

src/tasks.js

```
// CampusEats task list
const tasks = [
  "Design the menu screen",
  "Build the orders API",
  "Add user login",
];
console.log(`CampusEats has ${tasks.length} open tasks`);
```

3. Stage and commit your change with a clear, conventional message:

Terminal

```
$ git add src/tasks.js
$ git commit -m "feat: add initial CampusEats task list"
```

4. Push the branch to GitHub (the `-u` sets the upstream so later you can just `git push`):

Terminal

```
$ git push -u origin feature/add-task-list
```

Conventions to follow (from the lecture)

Branch naming	Commit messages (Conventional Commits)
feature/add-task-list	feat: add initial task list
fix/readme-typo	fix: correct typo in README
chore/add-ci	chore: add GitHub Actions CI

EXPECTED OUTPUT

A feature branch pushed to GitHub with at least one well-described commit. Screenshots of the branch list and the commit history, plus a one-line note explaining your branching strategy.

TASK 03 • 20 minutes • LO3

Pull Request and Code Review

A **pull request (PR)** proposes merging your branch into `main` and is the place where code review happens. The lecture's rule: changes are reviewed (and must pass checks) *before* they reach `main`.

Step-by-step instructions

1. On GitHub, you will see a prompt to `Compare & pull request` for your pushed branch — click it (or open the `Pull requests` tab → `New pull request`).
2. Set the base to `main` and the compare to `feature/add-task-list`. Write a clear title and a short description (what changed and why).
3. Do a code review. Working in pairs, review a classmate's PR (or self-review): open the `Files changed` tab, add at least one review comment, then `Approve` (or `Request changes`).
4. Merge the PR with `Merge pull request` (or `Squash and merge`), then click `Delete branch` to keep the repo tidy.
5. Bring the merged change back to your local `main`:

Terminal

```
$ git switch main
$ git pull origin main      # fast-forwards main with the merge
```

EXPECTED OUTPUT

A merged pull request with at least one review comment. Screenshots of the open PR (with its description), the review comment, and the merged/closed PR.

TASK 04 • 12 minutes • LO3

GitHub Issues and Collaborative Workflow

Teams plan and track work with **Issues** — small, labelled units of work that anyone can pick up. Issues link naturally to branches, commits, and pull requests.

Step-by-step instructions

1. Open the `Issues` tab → `New issue`. Create **three** issues for upcoming CampusEats work, for example:
 - “Add due dates to tasks” — label `enhancement`
 - “Fix typo in README” — label `bug` (or `documentation`)
 - “Add CI workflow” — label `enhancement`
2. For each issue, add a short description, a `label`, and (optionally) assign it to yourself.
3. Link work to an issue: a commit message or PR description that contains `Closes #1` will automatically close issue 1 when merged. Note this in your write-up.

EXPECTED OUTPUT

At least three labelled issues on the repository. A screenshot of the Issues list, and a note on how `Closes #<n>` links a commit or PR to an issue.

TASK 05 • 22 minutes • LO3, LO4

CI/CD Pipeline using GitHub Actions

Continuous Integration means every push is automatically checked. You will add a `GitHub Actions` workflow that runs on every push and pull request. Recall the lecture's pipeline: `Commit → Build → Test → Package → Deploy → Monitor`; today you automate the first stages.

Step-by-step instructions

1. On a new branch (`chore/add-ci`), create the file `.github/workflows/ci.yml` with this workflow:

```
.github/workflows/ci.yml
name: CI

on:
  push:
    branches: [ "main", "feature/**", "chore/**" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-check:
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repository
        uses: actions/checkout@v4

      - name: List repository files
        run: ls -la

      - name: Basic project check
        run: |
          echo "Running CI for CampusEats Task Tracker"
          test -f README.md && echo "README found"
```

2. Commit, push the branch, and open a pull request into `main`. The workflow starts automatically.

Terminal

```
$ git add .github/workflows/ci.yml
$ git commit -m "chore: add GitHub Actions CI workflow"
$ git push -u origin chore/add-ci
```

3. Open the `Actions` tab and watch the run. You should see a green check (✓). The same check also appears on your pull request — a failing check is what would **block a merge**.
4. Merge the PR once the check passes.

EXPECTED OUTPUT

A successful (green) GitHub Actions run triggered by your push/PR. Screenshots of the Actions run summary and the passing check shown on the pull request, plus your `ci.yml` file.

EXTEND THE PIPELINE (OPTIONAL)

To build a real project, replace the “Basic project check” step. For a **Node/React** repo (Lab 02): add `actions/setup-node@v4`, then `npm ci` and `npm test --if-present`. For a **.NET** repo (Lab 03): add `actions/setup-dotnet@v4`, then `dotnet build`.

TASK 06 • 13 minutes • LO3, LO4

Code Quality and Security Review

Finish like a professional team: review the code for quality and security. Below is a small function with several problems. Identify the issues, then commit an improved version.

Step 1 — Review this code and list its problems

```
src/tasks.js (before)
// BEFORE – what is wrong here?
function calc(a, b, t) {
  var x = a * b;
  if (t == "vip") { x = x - x * 0.1 }
  console.log("API_KEY=sk_live_9f8a7b6c5d"); // !!
  return x
}
```

List at least **three** problems. Look for: unclear names, a “magic number”, a hardcoded secret, `var` vs `const`, loose `==`, and missing input validation.

Step 2 — Commit a cleaner, safer version

```
src/tasks.js (after)
// AFTER – clear names, no magic numbers, no secrets
const VIP_DISCOUNT = 0.1;

function calculateTotal(price, quantity, customerType) {
  if (price < 0 || quantity < 0) {
    throw new Error("price and quantity must be >= 0");
  }
  const subtotal = price * quantity;
  return customerType === "vip"
    ? subtotal * (1 - VIP_DISCOUNT)
    : subtotal;
}
// the API key comes from an environment variable,
// e.g. process.env.API_KEY – never hard-coded
```

Step 3 — Check dependencies for security risks

1. If your repo has a `package.json`, run a dependency audit and note one finding and its fix:

Terminal

```
$ npm audit          # lists known-vulnerable packages
$ npm audit fix      # upgrades where a safe fix exists
```

2. Alternatively, open the repository’s `Security` tab and enable `Dependabot` alerts, which flags vulnerable dependencies automatically.

Step 4 — Reflection

Write 3–4 sentences on secure development practices you applied or learned: why secrets must never be committed, why dependencies need checking, and how code review and CI improve quality.

EXPECTED OUTPUT

A list of at least three code-quality issues, your improved code committed to the repo, a note from the dependency check (npm audit or Dependabot), and a short reflection on secure development.

THE GOLDEN SECURITY RULE

Never commit secrets (API keys, passwords, tokens) to Git. If a secret is ever pushed, treat it as **compromised**: remove it, **rotate** (regenerate) the key, and move it to an environment variable or secrets store. Deleting it in a later commit is *not* enough — it remains in the history.

QUICK TEST • 8 minutes**Knowledge Check**

Answer all questions in your Word document. Write the question number and your answer. Q1–Q5 are multiple choice; Q6 is Git-command-based; Q7 is CI/CD; Q8 is a security scenario.

Multiple choice (choose one)

Q1. Which command creates a new branch AND switches to it in one step?

- (a) git branch new
- (b) git checkout new
- (c) git switch -c new
- (d) git status -b new

Q2. A pull request is used to:

- (a) propose changes and have them reviewed before merging
- (b) delete a branch
- (c) install dependencies
- (d) run tests on your laptop

Q3. The main purpose of CI/CD is to:

- (a) design the database
- (b) style the user interface
- (c) write documentation
- (d) automatically build, test, and deploy changes

Q4. GitHub Actions workflow files are stored in:

- (a) the repository root
- (b) .github/workflows/ as .yml files
- (c) an /actions folder
- (d) a /ci folder

Q5. Which is a secure development practice?

- (a) commit API keys to the repo
- (b) hardcode the database password
- (c) store secrets in environment variables and never commit them
- (d) email passwords to the team

Git command-based

Q6. Write the two Git commands to stage all changes and commit them with the message `fix: correct total`.

CI/CD

Q7. Name any three stages of a typical CI/CD pipeline, in order.

Security scenario

Q8. A teammate pushes a commit that contains a live API key in `appsettings.json`. What is the risk, and what three actions should they take?

SUBMIT YOUR QUICK-TEST ANSWERS INSIDE THE SAME WORD DOCUMENT

Place your quick-test answers in a clearly labelled “Quick Test” section after your Task 01–06 answers.

Submission Instructions

Prepare a single Microsoft Word document containing all of your work for this lab, and a project ZIP if your lecturer requests it.

Your Word document must include:

- Student name
- IT number
- Lab practical number (Lab 08)
- Your GitHub repository link
- Screenshots of the Git commands you used
- Screenshots of the GitHub repository, branches, commits, and the pull request
- A screenshot of the GitHub Issues you created
- A screenshot of the GitHub Actions workflow execution (green run)
- Source-code snippets and your ci.yml workflow file
- A brief explanation of the branching strategy used
- A brief explanation of the code-quality improvements made
- A brief explanation of the security practices followed
- Your Quick Test answers (Q1–Q8)

File naming and upload:

1. Rename the Word document using your IT number: `ITXXXXXXXX_Lab08.docx`
2. Example: `IT22123456_Lab08.docx`
3. If submitting the project, name the ZIP: `ITXXXXXXXX_Lab08_Project.zip` (exclude `node_modules`, `bin`, `obj`).
4. Submit the Word document (and ZIP if required) to **CourseWeb** before the end of the 2-hour lab session.
5. **Late submissions will not be accepted** unless prior approval is given by the lecturer.

STUDENT INSTRUCTION

You must refer to the provided Lecture 08 material when completing the lab tasks. Your Git commands, workflow decisions, CI/CD configuration, code-quality improvements, and security explanations should be based on the lecture content, class explanations, and the practical examples discussed during the lecture.

Evaluation Criteria (Marking Guide — 20 Marks)

For lecturer reference. Marks reward a correct Git workflow, a working automated pipeline, and sound quality/security practice.

Component	Marks	What earns full marks
Task 01 — Repository setup	3	Public repo created and cloned; remote connected; git status / remote shown.
Task 02 — Branch & commit	3	Feature branch with a clear name; conventional commit message; branch pushed.

Component	Marks	What earns full marks
Task 03 — Pull request & review	3	PR opened with a description; at least one review comment; merged into main.
Task 04 — GitHub Issues	2	Three labelled issues; understands linking via Closes #n.
Task 05 — CI with GitHub Actions	4	Valid ci.yml triggered on push/PR; green run; explains how a failing check blocks a merge.
Task 06 — Quality & security	3	≥3 quality issues fixed; secret removed; dependency check + reflection.
Quick Test (Q1–Q8)	2	Q1–Q5 correct (c, a, d, b, c); Q6–Q8 show correct understanding.
Total	20	